Kaggle Challenge Report

WBC Classification

04 / 19 / 2026

Author : Baptiste GIRARDIN baptiste.girardin@telecom-paris.fr
School : **Télécom Paris**, France

Contents

I - Introduction . . . 3
I.1 - Objective . . . 3
I.2 - Key Challenges . . . 3
II - Explanations . . . 3
II.1 - Pre-processing . . . 3
II.1.a - Denoising . . . 3
II.1.b - Cell extraction . . . 3
II.2 - Image classification . . . 4
II.2.a - CNN Architecture . . . 4
II.2.b - Data Sampling . . . 6
II.2.c - Training Process . . . 6
II.3 - Output evaluation . . . 7
III - Conclusion . . . 7

Introduction

I.1 - Objective

The objective of this project was to be able to classify a dataset of **13 type** of white blood cells based on a training set of roughly **28 000 images**.

I.2 - Key Challenges

The main challenges of this project were the followings:

- The dataset had a lot of **noisy, blurred, darkened** images.
- Some classes were **very under-represented** (11 against $\sim$ 12000) and the evaluation metric was the macro-averaged F1 score :

$$F1_{\text{macro}} = \frac{1}{C} * \sum_{c=1}^{C} F1_c$$

Explanations

II.1 - Pre-processing

For this project, I did some offline pre-processing to properly segment the cells once for all. The main idea was to retrieve the purple shape at the center.

II.1.a - Denoising

Most of the images contained a lot of **speckle noise** as well as acquisition noise so I first applied a median filter using `cv2.medianBlur(image, ksize=9)` to remove the speckle noise, and then simplified the image the most I could using `cv2.edgePreservingFilter(median)`

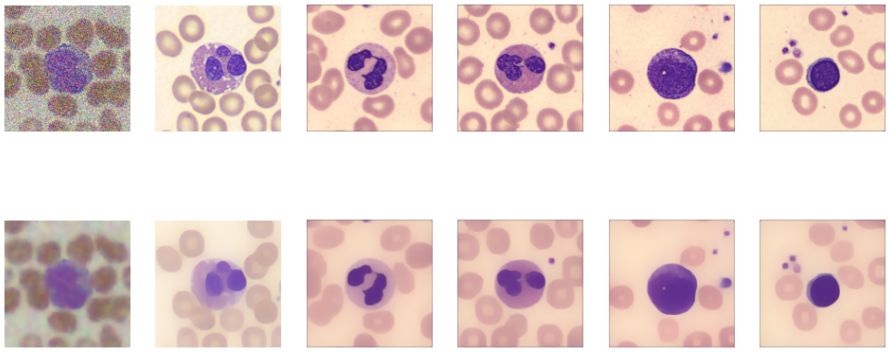

Figure 1: Cells without and with the denoising

II.1.b - Cell extraction

As all cells are at the center and have a purple shade, I roughly extracted a threshold mask using their hue channel in the HSV space.

Since sometimes some blood cells are also selected, I filter the result by using a connected component algorithm and selecting the one at the center : `cv2.connectedComponentsWithStats(mask)`.

Figure 2: Extracted cells

Sometimes this still doesn't work because another WBC or a blood cell is right next to the one of interest, so I try to separate them by:

- applying `ndimage.binary_fill_holes(mask)` to make sure the shape of interest is not affected by the next operation
- applying some erosion
- applying the connected component algorithm once again
- applying some dilation with a big disk (radius = 25) to compensate for the erosion and make sure to get the whole cell

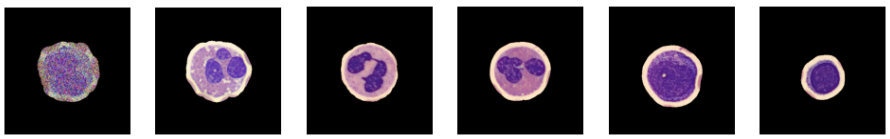

Figure 3: Extracted cells

II.2 - Image classification

For this step of the process, I decided to rely on a hand made and trained convolutional neural network to do some type of feature extraction as well as feature classification in one go. I didn't expect it to work very well because of my poor knowledge in CNNs, but I instead took this as an opportunity to learn.

II.2.a - CNN Architecture

First, for the inputs, I thought working on the RGB channels **and** on a channel representing the Local Binary Pattern (LBP) of the image would help the learning process as the cells granularity was an important factor for the classification and the convolutions tend to smooth the results, penalizing the granularity.

The architecture is as such :

RGB + LBP	→	encoder	→	classifier
256*256*4		512*8*8		13

Encoder

The encoder is composed of multiple layers of `nn.Conv2d(n, n, 3, stride=2, padding=1)` to compress more and more the image into a set of vectors characterizing it. Each layer is followed by :

- `nn.BatchNorm2d(n)`: To speed up and regularize the training process
- `nn.LeakyReLU(0.2)`: Acts like a ReLU layer, but prevents from the **dying ReLU** issue.
- `ResBlock(n)`: A hand-implemented class behaving like the block of a **Residual Network** to prevent from the **Vanishing Gradient** issue. This helps to make sure the CNN learns the details better by skipping the connections to the next convolutions.

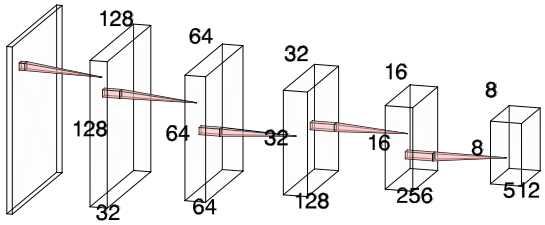

Figure 4: Encoder architecture

Classifier

Instead of doing an `nn.AdaptiveAvgPool2d(1)` and `nn.Flatten(1)` to compress it into a 1-dimension vector and classify it with an MLP, which increases the number of parameters drastically and looses spatial information, I chose to implement the idea behind *Network in Network*. Therefore I used `nn.Conv2d(n, n/2, 1)`, to reduce the length of the vector while keeping the spatial information. This leads to less overfitting, which is crucial when working with a class of only 11 representatives.

Each layer is followed by `nn.BatchNorm2d` and `nn.ReLU`, and finally I implemented a class `GeM()` to do **Generalized Mean Pooling**, in order to reduce the data to a single vector while keeping more information than `nn.AdaptiveAvgPool2d(1)`. This keeps the mean as in the Average Pooling method, but also keeps the maximum value.

Finally, I added a simple `nn.Linear(n, 13)` in the end to project the final vector to the classes.

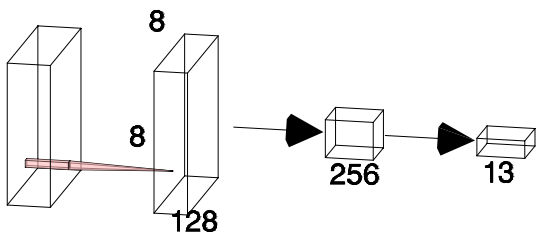

Figure 5: Classifier architecture

II.2.b - Data Sampling

The most difficult part of this challenge was to make sure that the model would learn the under-represented class while not overfitting them. To tackle this problem, I played on the key elements:

- The order / amount of sampling of each class
- The augmentation of each class

Sampling order and amount

For that I used pytorch's class `WeightedRandomSampler` which samples according to a given probability distributed among each images. A way to uniformly sample each class would be do give for each image the probability of the inverse of the cardinal of its class.

$$x_C \in C, p(x_C) = \frac{1}{|C|}$$

However, that way the model would just learn the few images of the under-represented classes and would therefore severely overfit. A more subtle oversampling probability would be

$$x_C \in C, p(x_C) = \frac{1}{(|C|)^\alpha}, \alpha \in [0, 1]$$

Here we don't care about $\sum_x p(x) > 1$ because `WeightedRandomSampler` automatically normalizes. I chose `0.3` for α because 11 is really small and the model would overfit at any higher value.

Data Augmentation

For this I used pytorch's transforms function. I compressed the images to 256*256, and them randomly flipped the images vertically and horizontally, as well as applying some random rotation from 0 to 30 degrees. I also applied some brightness variation and some speckle noise to mimic the dataset's deteriorations.

II.2.c - Training Process

For the training process I used:

- The `nn.CrossEntropyLoss(label_smoothing=0.1)` as the loss, since it is well adapted for classifications problems. `label_smoothing` is used to prevent any too overly confident prediction from the model
- The Adam optimizer
- An exponential scheduler to reduce exponentially the learning rate in order to avoid plateaus.

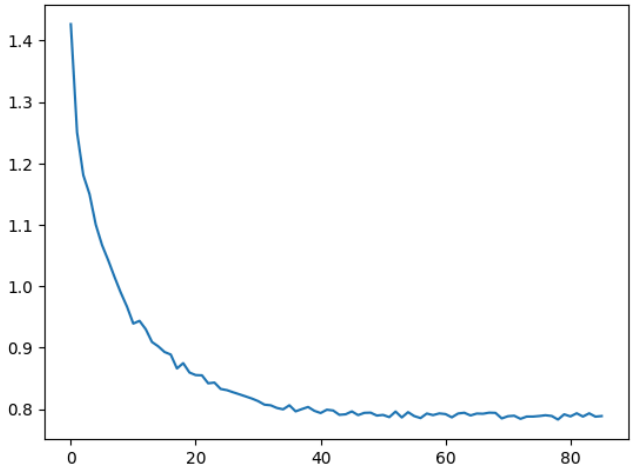

Figure 6: Evolution of the loss across the epochs

II.3 - Output evaluation

For this, I just used the most probable class given by the model for each image of the testing set.

Conclusion

This project showed how important good pre-processing is when working with noisy data. Extracting the cells beforehand made the classification task much easier.

On the model side, the custom CNN worked reasonably well despite the class imbalance. Using weighted sampling and data augmentation helped limit overfitting on rare classes, even if the problem remains difficult.

There is still room for improvement, especially with more advanced models or better tuning, but the results are solid given the constraints. Overall, it was a useful project to better understand both CNNs and the impact of data quality.